

Introducing Internet of Things (IoT) on AWS

Contents

Introducing Internet of Things (IoT) on AWS with a real-world scenario	1
Introduction	2
AWS IoT Core Activity for AWS Academy Learner Lab	2
Limitations of the Activity Setup	2
Scenario: Remote Home Temperature Monitoring and Control.....	2
AWS IoT Core in this Activity	4
AWS IoT Core Concepts in This Activity.....	4
Section 1: Setting Up AWS IoT Infrastructure	5
Section 2: Exploring Components	7
Section 3: Making the Smart Thermostat Work.....	7
Section 4: Simulation Temperature Readings	8
Section 5: Simulation Smartphone Controls	10
Summary	11
Topics to reflect and discuss with students	11
Further Reading	12

Introduction

The Internet of Things (IoT) refers to the interconnected network of physical devices, sensors, and software that collect and exchange data over the internet. As the demand for IoT skills grows, educators ask for practical ways to teach this technology effectively. This article explores how AWS Academy Learner Lab environment enable instructors to design hands-on, cloud-based lessons for teaching IoT concepts and practical skills using AWS IoT services.

AWS IoT Core: The Foundation of IoT on AWS

At the heart of AWS's IoT offerings lies the IoT Core service, which provides a secure and managed platform for connecting and managing IoT devices at scale. With IoT Core, students can learn how to provision and authenticate devices, ingest and process device data securely, and integrate IoT applications with other AWS services using rules and built-in integrations.

Within the Learner Lab, educators can create isolated virtual environments tailored to their teaching objectives. These environments provide a safe space for students to experiment with IoT Core features, such as device provisioning, data ingestion, and rule-based data processing, without the risk of impacting production systems or incurring unnecessary costs.

AWS IoT Core Activity for AWS Academy Learner Lab

Limitations of the Activity Setup

1. Our demo smart home system is simplified for learning.
2. The security policy is very open, allowing all actions. Real systems use stricter policies.
3. We're not using real hardware, just simulating with the MQTT client.
4. In a real system, you'd have more complex rules for processing and acting on data.
5. Production systems would include user authentication and device-specific permissions.

Scenario: Remote Home Temperature Monitoring and Control

Imagine you are part of a team developing a smart home thermostat system. This system will allow homeowners to monitor and control their home temperature remotely using their smartphones.

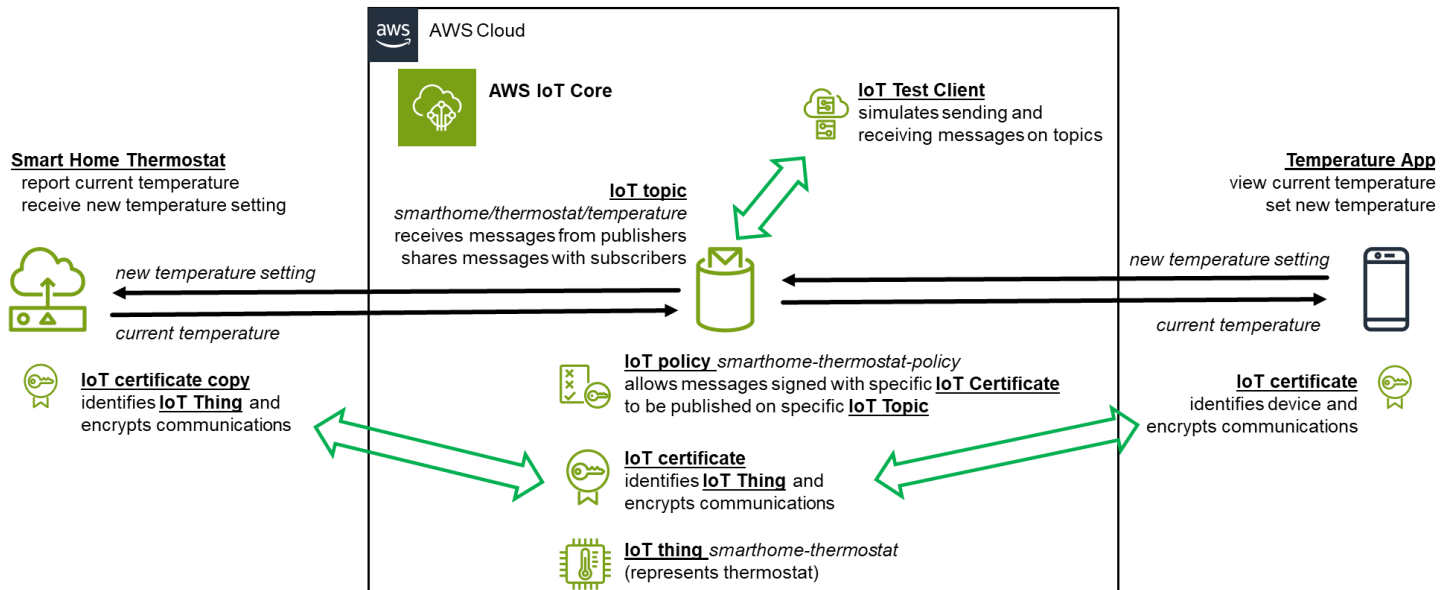
In this activity, you will create a simplified version of the cloud infrastructure that would support a real smart thermostat system. This setup allows for two-way communication:

- From the thermostat to the cloud (and then to your app) for monitoring.
- From your app to the cloud (and then to the thermostat) for control.

We'll use AWS IoT Core to simulate this smart thermostat setup.

Here's what we're building today using the AWS IoT Core services.

AWS IoT Scenario



This diagram illustrates:

1. An [IoT Thing](#) (representing our smart thermostat)
2. An [IoT Policy](#) (defining the thermostat's permissions)
3. The [Security Certificate](#), which is created manually and attached to both the IoT Thing and the IoT Policy.
4. The [MQTT Test Client](#) in the AWS Console, which we use to simulate both:
 - The thermostat publishing temperature data
 - A smartphone app subscribing to temperature data and publishing commands
5. [AWS IoT Core](#) managing the Thing, enforcing the Policy, and handling message routing.
6. A representation of how this relates to a real-world smart thermostat (which we're simulating with the MQTT client).

The objects inside the AWS Cloud portion of the diagram (IoT Thing, Certificate, Policy, etc.) represent the resources created by the CloudFormation and CloudShell commands you'll run.

This diagram gives a visual representation of our simulated smart home system, showing how the various components interact within AWS IoT Core and how they relate to the real-world devices we're simulating.

AWS IoT Core in this Activity

AWS IoT Core is a managed cloud service that allows connected devices to interact easily and securely with cloud applications and other devices. It provides:

- 1. Device connectivity and management**

AWS IoT Core handles the connection between your thermostat and the cloud. It ensures your device can connect securely and maintains the connection.

- 2. Secure data transmission**

The certificates and policies we set up ensure that all data sent between your thermostat, AWS IoT Core, and your smartphone app is encrypted and secure.

- 3. Message processing and routing**

When you send a command from your app, AWS IoT Core ensures it's routed correctly to your specific thermostat. Similarly, it routes temperature data from your thermostat to your app.

- 4. Integration with other AWS services**

While not covered in our basic scenario, AWS IoT Core could integrate with services like:

- AWS Lambda to trigger actions based on temperature (e.g., send an alert if it's too high)
- Amazon DynamoDB to store temperature history
- Amazon QuickSight to create visualizations of your home's temperature over time

AWS IoT Core Concepts in This Activity

- 1. Things: Virtual representations of devices**

In our setup, the "ThermostatThing" represents your smart thermostat. In AWS IoT Core, this "Thing" is a virtual representation of your physical thermostat device. It allows AWS to recognize and manage your thermostat in the cloud.

- 2. Topics: Channels for publishing/subscribing to messages**

We use two MQTT topics in our scenario:

- "smarthome/thermostat/temperature": This is where your thermostat would publish its temperature readings. In a real setup, your smartphone app would subscribe to this topic to display current temperature.
- "smarthome/thermostat/commands": This is where your smartphone app would publish commands. Your thermostat would subscribe to this topic to receive instructions like temperature adjustments.

- 3. Certificates: Ensure secure device communication**

We create and attach a certificate to our virtual thermostat. In a real-world scenario, your physical thermostat would have a similar certificate. This ensures that only your authentic thermostat can send data or receive commands, protecting against unauthorized access.

- 4. MQTT: A lightweight messaging protocol used for device communication**

This is the protocol we use for sending messages. MQTT is ideal for IoT devices because it's lightweight and efficient. In our scenario:

- Your thermostat would use MQTT to send temperature data to AWS IoT Core.
- Your smartphone app would use MQTT (via AWS IoT Core) to send commands to your thermostat.

Section 1: Setting Up AWS IoT Infrastructure

Prerequisites

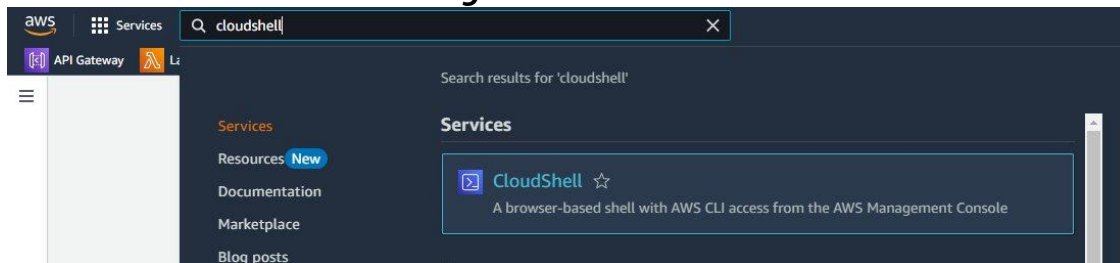
Each student has access to an AWS Academy Learner Lab.

Setting Up the Virtual Thermostat

These steps create a virtual representation of your smart thermostat in AWS. In a real-world scenario, this would correspond to registering your physical thermostat with a cloud service.

Step 1: Navigate to AWS Management Console.

Step 2: Open CloudShell in the AWS Management Console



Step 3: Create the Configuration File

- In CloudShell, create a new file by typing: `nano smart-thermostat.yaml`
- Copy and paste the provided YAML content into this file:

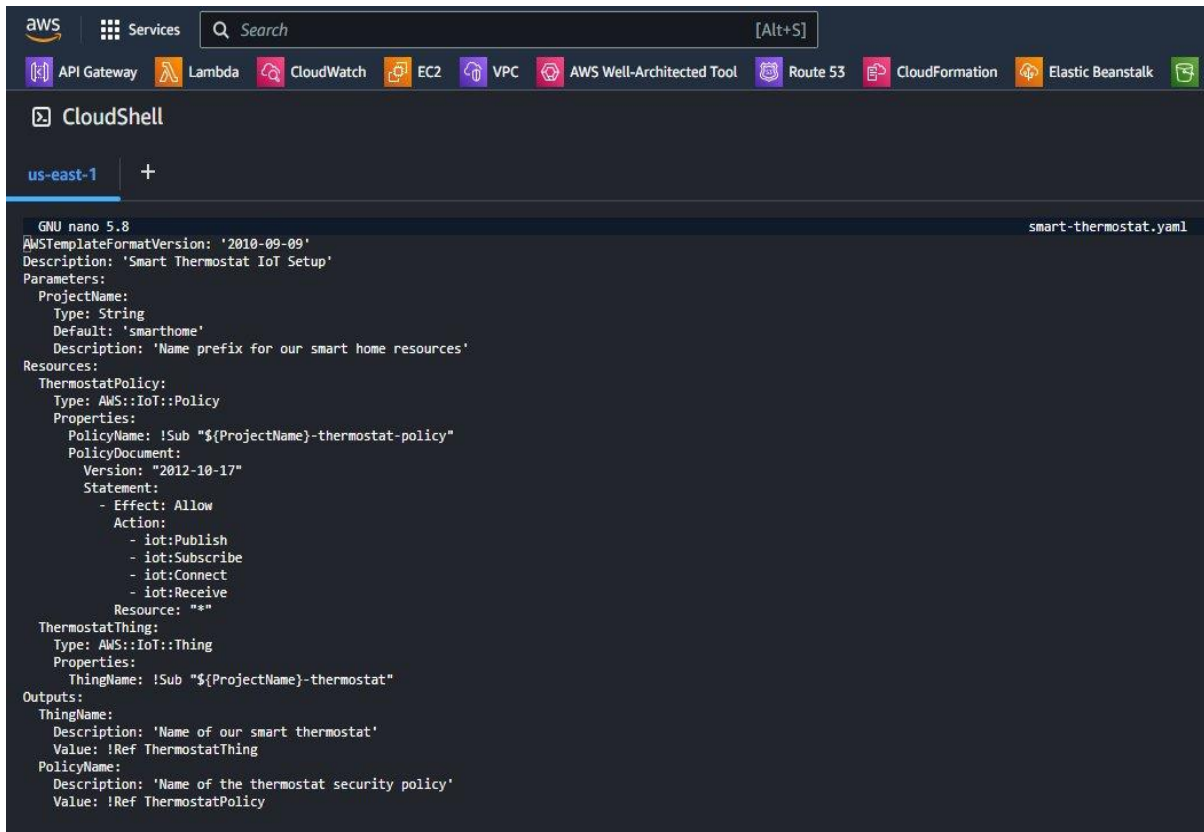
```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Smart Thermostat IoT Setup'
Parameters:
  ProjectName:
    Type: String
    Default: 'smarthome'
    Description: 'Name prefix for our smart home resources'
Resources:
  ThermostatPolicy:
    Type: AWS::IoT::Policy
    Properties:
      PolicyName: !Sub "${ProjectName}-thermostat-policy"
      PolicyDocument:
        Version: "2012-10-17"
        Statement:
          - Effect: Allow
            Action:
              - iot:Publish
              - iot:Subscribe
```

```

        - iot:Connect
        - iot:Receive
    Resource: "*"
ThermostatThing:
  Type: AWS::IoT::Thing
  Properties:
    ThingName: !Sub "${ProjectName}-thermostat"
Outputs:
  ThingName:
    Description: 'Name of our smart thermostat'
    Value: !Ref ThermostatThing
  PolicyName:
    Description: 'Name of the thermostat security policy'
    Value: !Ref ThermostatPolicy

```

- Save the file by pressing Ctrl+X, then Y, then Enter:



The screenshot shows the AWS CloudShell interface. At the top, there's a navigation bar with various AWS services icons. Below that, the CloudShell title bar shows 'us-east-1' and a plus sign. The main area displays the content of a file named 'smart-thermostat.yaml' in a nano editor. The content is an AWS CloudFormation template for a smart thermostat setup, including parameters for ProjectName, resources for ThermostatPolicy and ThermostatThing, and outputs for ThingName and PolicyName.

```

GNU nano 5.8 smart-thermostat.yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Smart Thermostat IoT Setup'
Parameters:
  ProjectName:
    Type: String
    Default: 'smarthome'
    Description: 'Name prefix for our smart home resources'
Resources:
  ThermostatPolicy:
    Type: AWS::IoT::Policy
    Properties:
      PolicyName: !Sub "${ProjectName}-thermostat-policy"
      PolicyDocument:
        Version: "2012-10-17"
        Statement:
          - Effect: Allow
            Action:
              - iot:Publish
              - iot:Subscribe
              - iot:Connect
              - iot:Receive
            Resource: "*"
  ThermostatThing:
    Type: AWS::IoT::Thing
    Properties:
      ThingName: !Sub "${ProjectName}-thermostat"
Outputs:
  ThingName:
    Description: 'Name of our smart thermostat'
    Value: !Ref ThermostatThing
  PolicyName:
    Description: 'Name of the thermostat security policy'
    Value: !Ref ThermostatPolicy

```

Step 4: Deploy the Smart Home Setup

- Run the following command in CloudShell:

```
aws cloudformation create-stack --stack-name smart-thermostat --
template-body file://smart-thermostat.yaml
```

- Wait for the command to complete. It will return a StackId if successful.

Step 5: Check the Setup Status

- Run the following command to check if the setup is ready:

```
aws cloudformation describe-stacks --stack-name smart-thermostat
```

- Look for "StackStatus" in the output. If it says "CREATE_COMPLETE", your setup is ready.

Section 2: Exploring Components

This step shows you where to find your virtual thermostat and its security policy in AWS. In reality, this would be handled by the thermostat manufacturer, but it's useful to understand these components.

Step 1: Navigate in the AWS Management Console to "IoT Core".

Step 2: To view the Smart Thermostat:

- In IoT Core, click on "Manage" in the left sidebar, then "All devices" and "Things".
- Look for an item named "smarthome-thermostat".
-

Step 3: To view the Thermostat Security Policy:

- In IoT Core, click on "Security" in the left sidebar, then "Policies".
- Look for a policy named "smarthome-thermostat-policy".

You can see that the policy allows the thermostat to connect to AWS IoT Core, publish data (like temperature readings), subscribe to topics (for receiving commands), and receive messages.

Important: The policy we've created allows broad permissions for demonstration purposes. In a real IoT system, you'd create more fine-grained policies specific to each device and its required actions.

Section 3: Making the Smart Thermostat Work

Securing Communication

This step sets up secure communication between your virtual thermostat and AWS by creating and attaching a security certificate. In a real setup, this ensures that only your thermostat can send data and receive commands, protecting your privacy and security.

Steps: In CloudShell, run these commands:

- Create a new certificate for your IoT device (the thermostat), activate it, and store its Amazon Resource Name (ARN) in a variable for later use:

```
CERT_ARN=$(aws iot create-keys-and-certificate --set-as-active --query 'certificateArn' --output text)
```
- Associate the newly created certificate with your virtual thermostat, allowing the thermostat to use this certificate for secure communication:

```
aws iot attach-thing-principal --thing-name smarthome-thermostat --principal $CERT_ARN
```
- Attach the security policy we created earlier to the certificate, defining what actions the thermostat is allowed to perform when communicating with AWS IoT Core:

```
aws iot attach-policy --policy-name smarthome-thermostat-policy --target $CERT_ARN
```

Section 4: Simulation Temperature Readings

This simulates your thermostat sending temperature data to the cloud using the AWS IoT MQTT test client. In reality, your physical thermostat would periodically send real temperature and humidity readings. This data would then be accessible through your smartphone app.

Step 1: In IoT Core, click on “Test” in the left sidebar, then “MQTT test client”.

Note: In the IoT Core test client, you need to subscribe to a topic before publishing to it. Any messages sent before subscribing are not shown.

Step 2. Subscribe to temperature topic

- In the MQTT test client click on “Subscribe to a topic”
- In the “Topic Filter” field, enter “smarthome/thermostat/temperature” without quotes.
- Click the Subscribe button. You will see the Message Payload box at the bottom of the screen populate.

Step 3: In the “Publish to a topic” section

- On the MQTT Test Client screen, click “Publish to a topic”.
- For “Topic Name”, enter “smarthome/thermostat/temperature” without quotes.
- For “Message”, enter {"temperature": 72, "humidity": 50, "timestamp": "2023-11-15T10:00:00Z"}
- Click “Publish” to send the message. You will see the value in the Message Payload box of the previous step.

Note: You can also publish to a subscribed topic using the Subscriptions area after selecting a topic you’re subscribed.

After publishing to a subscribed topic, you'll see the message data. One value of using MQTT is that a topic can have multiple subscribers who all receive the same data at the same time. In a real-world scenario, this data could be received by both your cloud application and another smart monitor in your home.

Keep in mind: In this demo, we're manually sending data through the MQTT client. In a real system, your physical thermostat would automatically send this data, and you'd have more complex rules for processing and acting on this data.

Section 5: Simulation Smartphone Controls

This step mimics you using your smartphone app to adjust the thermostat. In a real scenario:

- You would open your thermostat app on your phone.
- You'd set a new desired temperature, say 70°F.
- The app would send this command to the cloud (simulated by our MQTT publish step).
- The cloud service would then relay this command to your physical thermostat at home.
- Your thermostat would adjust to reach the new temperature.

Steps: In the same MQTT test client:

1. In the "Subscribe to a topic" section:
 - a. Enter "smarthome/thermostat/commands"
 - b. Click "Subscribe"
2. In the "Publish" section:
 - a. For "Topic", enter "smarthome/thermostat/commands"
 - b. For "Message", enter: {"action": "set_temperature", "value": 70}
3. Click "Publish" to send the command.

To see the result, look at the "Subscriptions" section of the MQTT test client. You should see the message you just published appear here, simulating how a real thermostat would receive the command.

If you see the error "Message cannot be displayed in specific format" that can sometimes happen with copy-pasting from this document. In that case, type the message in the top "message payload" box as shown here:

The topic name identifies the message. The message payload will be published to this topic with a Quality of Service (QoS) of 0.

Q smarthome/thermostat/commands X

Message payload

```
{
  "command": "set-temperature", "value": 70
}
```

► Additional configuration

Publish

Subscriptions smarthome/thermostat/commands [Pause] [Clear] [Export] [Edit]

smarthome/thermostat/commands ❤ X

Message payload

```
{
  "message": "Hello from AWS IoT console"
}
```

► Additional configuration

Publish

▼ smarthome/thermostat/commands November 27, 2024, 08:01:36 (UTC-0500)

```
{
  "command": "set-temperature",
  "value": 70
}
```

Note: While we're simulating smartphone control here, a production system would include user authentication to ensure only authorized users can control the thermostat. Additionally, you'd have device-specific permissions to ensure each user can only control their own devices.

Note: This is an example. In a real-world scenario, certain device settings are managed using the [AWS IoT Device Shadow](#), which tracks and compares a device's requested and current state.

Summary

In this activity, students created a simplified smart home thermostat system using AWS IoT Core within the AWS Academy Learner Lab environment. They set up a virtual IoT device (thermostat), configured security policies, and simulated both temperature data transmission and remote control commands using MQTT protocol. Through this hands-on exercise, students gained practical experience with key IoT concepts such as device representation, secure communication, and message routing in a cloud-based IoT infrastructure, while also understanding the differences between this simplified setup and real-world IoT implementations.

Topics to reflect and discuss with students

1. Security Considerations
 - a. How might the security policy need to be modified for a production environment?
 - b. What additional security measures would be necessary for a real smart home system?

2. Scalability
 - a. How would this system need to change to support thousands or millions of devices?
 - b. What might need to change for this system to work in a large building like a school with multiple rooms and thermostats?
3. Data Management
 - a. In a real-world scenario, how would you handle and store the large amounts of data generated by IoT devices?
 - b. What kinds of insights could be derived from analyzing temperature data over time?
4. User Experience
 - a. How would you design a user-friendly mobile app interface for controlling the smart thermostat?
 - b. What additional features might users want in a smart home system?
5. Future Developments
 - a. How might emerging technologies like AI and machine learning be integrated into smart home systems?
 - b. What could be possible if more devices in a home could share data?

Further Reading

1. [AWS IoT for Smart Home] shows specific examples of how AWS IoT is being used for smart home applications. (<https://aws.amazon.com/iot/solutions/connected-home/>)
2. [IoT Security Best Practices] outlines security best practices for AWS IoT. (<https://docs.aws.amazon.com/iot/latest/developerguide/security-best-practices.html>)
3. [AWS IoT Blog] AWS IoT service announcements and examples are regularly published here. (<https://aws.amazon.com/blogs/iot/>)
4. [Case Study] present how Volkswagen Group and Porsche built a connected factory using AWS IoT Core. (<https://aws.amazon.com/solutions/case-studies/volkswagen-group-case-study>)
5. [Case Study] shows how IoT Core contributed to Philips HealthSuite Platform speeding Healthcare Innovation on AWS. (<https://aws.amazon.com/solutions/case-studies/philips-healthsuite-digital-platform-case-study>)